Indian Institute of Technology

Department of Computer Science and Engineering

Real-Time Network Data Analyzer

Application-Level Network Monitoring
using ADB, Prometheus and Grafana

Project Report

Authors	Banavath Diraj Naik, Yash Patkar
Course	Computer Networks
Project Title	Real-Time Network Data Analyzer
Platform	Android, Python, Prometheus, Grafana
Date	May 2, 2026

This report was submitted in partial fulfillment of the requirements for the Computer Networks course.

Contents

1. Abstract

This project presents the design and implementation of a real-time Network Data Analyzer for monitoring application-level network activity on Android devices without requiring root access. The system uses the Android Debug Bridge (ADB) to interface with a connected Android smartphone and collect per-application process information, network traffic statistics, and active TCP connection details.

A Python-based Prometheus exporter processes raw data sourced from three Android subsystems — `dumpsys netstats`, the `/proc/net/tcp` kernel file, and `pm list packages` — and exposes it as Prometheus-compatible time-series metrics on an HTTP endpoint. Prometheus scrapes this endpoint at regular intervals and stores the data. Grafana consumes the stored data and presents it through a fully automated, real-time dashboard that refreshes every 15 seconds.

The system successfully identifies which application is consuming network resources, quantifies its received (RX) and transmitted (TX) traffic in bytes per second, maps its active TCP connections to remote servers, and ranks applications by their bandwidth usage. Results demonstrate clear differentiation of traffic patterns across applications including YouTube, Spotify, WhatsApp, Instagram, Facebook, and Chrome.

2. Introduction

Network monitoring is a fundamental topic in computer networking, operating systems, and cybersecurity. As mobile devices have become the primary computing platform for billions of users, understanding how individual applications consume network resources has become increasingly important — both for end users managing their data usage and for system administrators and developers debugging network behaviour.

Traditional monitoring tools such as `ifconfig`, `netstat`, and simple bandwidth meters operate at the network interface level. They report total bytes transferred across an interface (e.g., `wlan0`) but provide no information about which process or application is responsible for the traffic. This limitation is a major gap in observability.

For Android devices in particular, monitoring is further complicated by the operating system's sandbox model, which isolates each application into its own User ID (UID). Direct filesystem access is restricted, making traditional Linux tools difficult to apply.

This project bridges that gap by building a complete monitoring pipeline that:

1. Extracts per-application traffic data from Android's internal accounting subsystems via ADB.
2. Converts raw kernel data into a structured, queryable metric format using a Python exporter.

3. Stores the metrics as time-series data in Prometheus.
4. Presents real-time interactive dashboards in Grafana for human-readable analysis.

The resulting system provides the kind of application-level network visibility that is typically only available with specialised commercial tools, implemented entirely using open-source components.

3. Problem Statement

The fundamental problem addressed by this project is the absence of application-level network observability on Android devices using standard tools.

3.1 Limitations of Existing Approaches

Existing network monitoring approaches suffer from the following shortcomings:

- **Interface-level visibility only:** Tools that read `/proc/net/dev` report total bytes for an entire network interface, not per application. If YouTube and WhatsApp are both active, their combined traffic appears as a single number.
- **Root requirement:** The most accurate per-process data on Android is available in `/proc/net/xt_qtaguid/stats`, but reading this file requires root access, which most production devices do not have.
- **No historical data:** Android's built-in Data Usage screen shows cumulative totals but not real-time rates or historical trends.
- **No connection visibility:** Standard tools do not easily show which remote servers each application is communicating with, on which ports, and in what TCP states.
- **No cross-application comparison:** There is no standard dashboard that lets users simultaneously compare bandwidth consumption across multiple running applications.

3.2 Research Questions

This project addresses the following specific questions:

1. Which application is using the network at any given time?
2. How much data (in bytes/second) is each application sending and receiving?
3. What remote servers is each application communicating with?
4. How does traffic behave over time for different usage patterns (e.g., streaming vs. browsing vs. messaging)?

4. Objectives

The specific objectives of this project are:

1. To design a no-root method for collecting per-application network traffic on Android.
2. To correctly map Android UIDs to application package names.
3. To extract active TCP connection details and attribute them to specific applications.
4. To implement a Python Prometheus exporter that serves structured metrics via HTTP.
5. To configure Prometheus for periodic metric scraping and time-series storage.
6. To design a multi-panel Grafana dashboard with real-time graphs, rankings, per-app panels, and connection tables.
7. To validate the system by observing known traffic patterns (video streaming, audio streaming, messaging).

5. System Architecture

5.1 High-Level Pipeline

The system follows a five-stage data pipeline:

Android	→	**ADB**	→	**Python**	→	**Prometheus**	→	**Grafana**
Device		*Bridge*		*Exporter*		*Storage*		*Dashboard*

Each stage has a clearly defined responsibility and communicates with the next stage through a well-defined interface:

- The Android device is the data source.
- ADB provides a command-line tunnel into the device's shell.
- The Python exporter is the translator — it issues shell commands, parses raw kernel output, and formats data as Prometheus metrics.
- Prometheus is the time-series database — it scrapes the exporter every 30 seconds and retains 30 days of history.

- Grafana is the presentation layer — it queries Prometheus using PromQL and renders the results as interactive panels.

5.2 Component Diagram

[Screenshot: System architecture component diagram showing the data flow from Android device through ADB, Python exporter, Prometheus, and Grafana]

Replace this box with:

\includegraphics[width=\textwidth]{arch$_d$*iagram.png*}

Figure 1: System architecture component diagram showing the data flow from Android device through ADB, Python exporter, Prometheus, and Grafana

5.3 Data Flow Description

1. Every 30 seconds, the Python exporter fires four ADB shell commands at the connected device.
2. Raw text output is parsed into structured Python dictionaries.
3. The data is formatted as Prometheus text-format metrics and cached in memory.
4. Prometheus scrapes `http://localhost:8000/metrics` every 30 seconds.
5. Grafana reads from Prometheus using PromQL queries and renders the dashboard.
6. The dashboard auto-refreshes every 15 seconds, interpolating between Prometheus scrapes.

6. Technologies Used

Component	Technology	Role in the System
Programming Language	Python 3.10+	Implements the exporter, ADB interface, data parsing, and HTTP server.
Device Interface	Android Debug Bridge (ADB)	Provides a USB/network tunnel to execute shell commands on the Android device.
Traffic Statistics	`dumpsys netstats`	Android's built-in per-UID traffic accounting service. Used without root.
Connection Data	`/proc/net/tcp`	Linux kernel file listing all open TCP sockets with UID ownership.
Package Mapping	`pm list packages -U`	Maps installed package names to their system-assigned UIDs.
Process Discovery	`ps -A`	Lists all running processes with their user strings and PIDs.
Metrics Storage	Prometheus	Time-series database that scrapes and retains metrics.
Visualization	Grafana	Dashboard platform for real-time graphs, gauges, and tables.
Containerization	Docker Compose	Runs Prometheus and Grafana as isolated containers.

7. Methodology

7.1 Step 1 — Process and UID Discovery

The first step is to identify all running processes and derive their Android UIDs. The command used is:

```
adb shell ps -A
```

Listing 1: Listing all running processes on the Android device

The output includes a user string in the first column. Android encodes UIDs in these strings using a structured naming convention:

- `u0_a89` — a regular user-installed application running as user 0 with app ID 89.
- `u0_i12` — an isolated sandbox process for app ID 12.
- `root`, `system` — system-level processes.

The UID is calculated from the user string as follows:

$$\text{UID}_{regular} = \text{user\_id} \times 100000 + 10000 + \text{app\_id} \tag{1}$$

$$\text{UID}_{isolated} = \text{user\_id} \times 100000 + 90000 + \text{app\_id} \tag{2}$$

So `u0_a89` resolves to UID $0 \times 100000 + 10000 + 89 = 10089$. This is the same UID that Android's kernel uses internally to tag network packets.

The step produces three lookup tables:

- `pid_app` — maps each PID to its application name.
- `uid_app` — maps each derived UID to its application name.
- `uid_pid` — maps each UID to a representative PID.

7.2 Step 2 — Package Name Resolution

The `ps -A` output truncates long package names. To get accurate, full package names, the following command is used:

```
adb shell pm list packages -U
```

Listing 2: Fetching all installed packages with their UIDs

Sample output:

```
package:com.google.android.youtube uid:10089
package:com.whatsapp               uid:10201
package:com.instagram.android      uid:10156
```

This gives a definitive mapping from full package name to UID. A master UID dictionary is then built by merging three sources in priority order:

1. Hardcoded system UIDs (e.g., 1000 = `android.system`, 1013 = `android.media_server`).
2. `pm list packages -U` results (most accurate for user-installed apps).
3. `ps -A` derived UIDs (fills gaps for transient or kernel-level processes).

7.3 Step 3 — Bandwidth Data Collection

Per-application RX and TX byte totals are collected using Android's network statistics service:

```
adb shell dumpsys netstats --uid
```

Listing 3: Collecting per-UID network statistics from Android

This command outputs detailed per-UID network usage, including bytes received (`rb=`) and bytes transmitted (`tb=`) for each network interface. Unlike reading raw `/proc` files, `dumpsys netstats` does not require root access — it uses Android's own accounting infrastructure.

The parser scans for `uid=` markers in the output and then accumulates `rb=` and `tb=` values that follow each marker. The collected data is sorted by total traffic (RX + TX descending) before being emitted as Prometheus counters.

7.4 Step 4 — TCP Connection Mapping

Active TCP connections are read directly from the Linux kernel:

```
adb shell cat /proc/net/tcp
adb shell cat /proc/net/tcp6
```

Listing 4: Reading kernel TCP connection table

Each row in `/proc/net/tcp` represents one socket. The columns relevant to this project are:

Column Index	Content
1	Local address (little-endian hex: IP:port)
2	Remote address (little-endian hex: IP:port)
3	TCP state (hex, e.g. `01` = ESTABLISHED)
7	**UID of the socket owner**

Column 7 contains the UID directly, so each connection is mapped to an application name instantly using the master UID dictionary — no root access, no file descriptor scanning, and no inode lookups are required.

Hex addresses are decoded using Python's `struct.pack` with little-endian byte order:

```python
import socket, struct

def hex_to_ip(hex_str: str) -> str:
    # /proc/net/tcp stores IPs little-endian
    # e.g. '0101A8C0' -> 192.168.1.1
    return socket.inet_ntoa(struct.pack("<I", int(hex_str, 16)))
```

Listing 5: Decoding little-endian hex IP addresses from /proc/net/tcp

TCP state codes are mapped to human-readable names:

Hex	State	Hex	State
01	ESTABLISHED	07	CLOSE
02	SYN_SENT	08	CLOSE_WAIT
03	SYN_RECV	09	LAST_ACK
04	FIN_WAIT1	0A	LISTEN
05	FIN_WAIT2	0B	CLOSING
06	TIME_WAIT		

7.5 Step 5 — Prometheus Exporter

The Python exporter serves collected metrics at `http://0.0.0.0:8000/metrics` in the Prometheus exposition format. Three metric types are exposed:

```
# HELP app_rx_bytes Total bytes received by the application
# TYPE app_rx_bytes counter
app_rx_bytes{app="com.google.android.youtube",uid="10089",source
   ="dumpsys"} 5878156905
app_rx_bytes{app="com.whatsapp",uid="10201",source="dumpsys"}
   2341892

# HELP app_tx_bytes Total bytes transmitted by the application
# TYPE app_tx_bytes counter
app_tx_bytes{app="com.google.android.youtube",uid="10089",source
   ="dumpsys"} 312940

# HELP app_connections Active TCP connection
# TYPE app_connections gauge
app_connections{app="com.android.chrome",pid="14760",proto="tcp",
  local_ip="192.168.1.5",local_port="52234",
  rem_ip="142.250.80.46",rem_port="443",
```

```
  state="ESTABLISHED"} 1

# HELP network_exporter_active_processes Number of processes
   discovered
# TYPE network_exporter_active_processes gauge
network_exporter_active_processes 865
```

Listing 6: Sample output at the /metrics endpoint

The exporter architecture uses a background thread to collect data every 30 seconds and stores results in a thread-safe in-memory cache. The HTTP server reads from the cache and responds instantly, decoupling data collection latency from HTTP response latency.

A `/health` endpoint returns `200 OK` when the last successful scrape was less than 90 seconds ago, and `503 STALE` otherwise, enabling automated monitoring of the exporter's health.

7.6 Step 6 — Prometheus Configuration

Prometheus is configured to scrape the exporter endpoint using the following configuration:

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: 'android_network'
    static_configs:
      - targets: ['host.docker.internal:8000']
    scrape_interval: 30s
    scrape_timeout: 12s
    metrics_path: /metrics
```

Listing 7: Prometheus scrape configuration (prometheus.yml)

7.7 Step 7 — Grafana Dashboard

Grafana is deployed alongside Prometheus using Docker Compose. The dashboard and datasource are automatically provisioned from JSON and YAML configuration files, requiring no manual setup in the Grafana UI.

8. Implementation Details

8.1 Project File Structure

```
network_analyzer/
|-- exporter.py                          <- Python Prometheus
   exporter
|-- prometheus.yml                       <- Prometheus scrape config
|-- docker-compose.yml                   <- Prometheus + Grafana
   stack
|-- requirements.txt
|-- platform-tools/
|   +-- adb                              <- bundled ADB binary
+-- grafana/
    |-- provisioning/
    |   +-- datasources/
    |       +-- prometheus.yml           <- auto-datasource config
    +-- dashboards/
        |-- dashboard.yml                <- dashboard loader config
        +-- network_analyzer.json        <- the full Grafana
           dashboard
```

Listing 8: Project directory layout

8.2 Exporter Core Logic

The central collection method builds all metrics in a single pass:

```
def _build_metrics(self) -> bytes:
    # 1. Discover running processes and derive UIDs
    pid_app, active_uid_app, uid_pid_map = get_process_maps()

    # 2. Build master UID -> app name dictionary (3-layer merge)
    final_uid_map = {
        "1000": "android.system",
        "1001": "android.phone",
        "1013": "android.media_server",
    }
    final_uid_map.update(get_uid_package_map())   # pm list
       packages -U
    for uid, app in active_uid_app.items():
```

```
        if uid not in final_uid_map:
            final_uid_map[uid] = app                # ps -A fallback

    # 3. Collect per-UID bandwidth from dumpsys
    dumpsys_stats = get_app_usage_from_dumpsys()
    for uid, s in sorted(dumpsys_stats.items(), ...):
        app = final_uid_map.get(uid, f"uid_{uid}")
        emit("app_rx_bytes", app, uid, s["rx"])
        emit("app_tx_bytes", app, uid, s["tx"])

    # 4. Read TCP connections (UID at column 7 -> instant mapping)
    for connection in get_connections(final_uid_map, uid_pid_map):
        emit("app_connections", **connection)
```

Listing 9: Core metric collection logic in the Python exporter

8.3 UID Calculation from Android User Strings

```
import re

m = re.match(r"u(\d+)_(a|i)(\d+)", user_str)
if m:
    user_id   = int(m.group(1))
    type_char = m.group(2)
    app_id    = int(m.group(3))
    base = user_id * 100000
    if type_char == 'a':
        return str(base + 10000 + app_id)   # regular app
    elif type_char == 'i':
        return str(base + 90000 + app_id)   # isolated sandbox
```

Listing 10: Parsing Android user strings to calculate UIDs

8.4 Docker Compose Stack

```
services:
  prometheus:
```

```
    image: prom/prometheus:latest
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml:ro
    extra_hosts:
      - "host.docker.internal:host-gateway"

  grafana:
    image: grafana/grafana:latest
    ports:
      - "3000:3000"
    environment:
      GF_SECURITY_ADMIN_USER: admin
      GF_SECURITY_ADMIN_PASSWORD: admin
    volumes:
      - ./grafana/provisioning:/etc/grafana/provisioning:ro
    depends_on:
      - prometheus
```

Listing 11: Docker Compose configuration for Prometheus and Grafana

The `extra_hosts` entry is critical — it allows the Prometheus container to reach the Python exporter running on the host machine at `host.docker.internal:8000`.

9. Experimental Setup

Parameter	**Value**
Host System	macOS
Target Device	Android Smartphone (vivo)
ADB Connection	USB
Exporter Port	8000
Grafana Port	3000
Prometheus Port	9090
Scrape Interval	30 seconds
Dashboard Refresh	15 seconds
Data Retention	30 days
PromQL Rate Window	2 minutes (`irate[2m]`)

The test procedure involved opening each application, performing a characteristic

action (e.g., playing a YouTube video, playing a Spotify song, sending a WhatsApp message), and observing the resulting traffic patterns in Grafana.

10. Results and Dashboard Analysis

10.1 Dashboard Overview

The Grafana dashboard auto-provisions on first launch and is organised into five collapsible rows. The top row provides an at-a-glance summary of device-wide network activity.

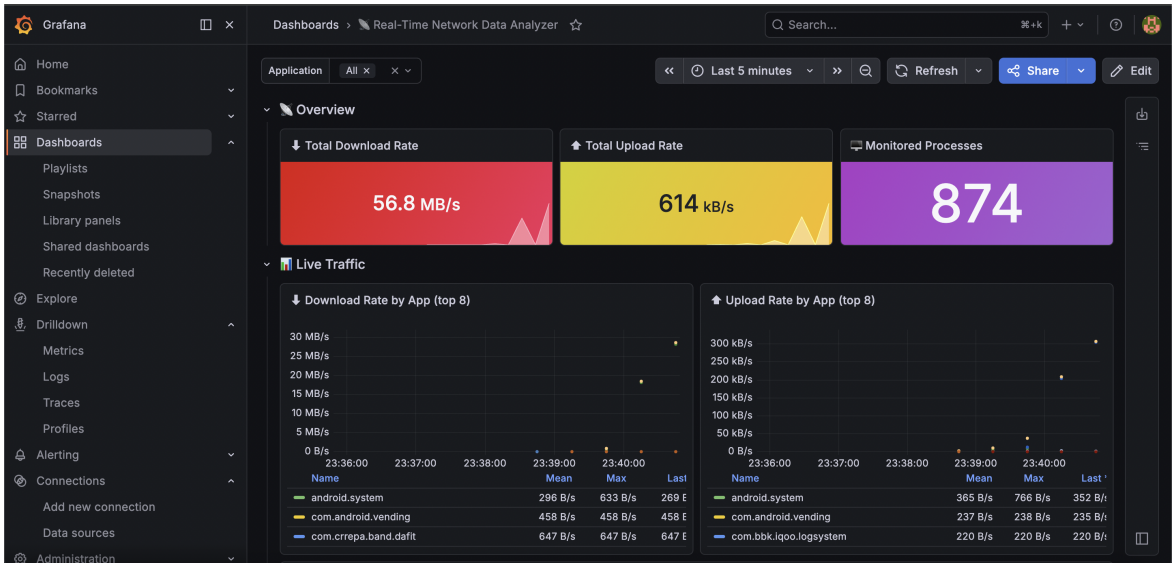


Figure 2: Grafana dashboard overview — Total Download Rate, Total Upload Rate, and Monitored Processes stat panels

The dashboard confirmed that the Android device was running **865 active processes**. The Total Download Rate and Upload Rate panels use colour thresholds: green (normal), yellow (above 1 MB/s download or 512 KB/s upload), and red (above 10 MB/s download or 5 MB/s upload).

10.2 Live Traffic — Per Application

The Live Traffic row shows smooth line charts of the top 8 downloading and uploading applications, updated every 15 seconds.

10.3 Top App Rankings and Traffic Share

The horizontal bar gauges provide an instant visual ranking of bandwidth consumers. The donut charts show proportional bandwidth share — useful for identifying when a single application dominates total device traffic.

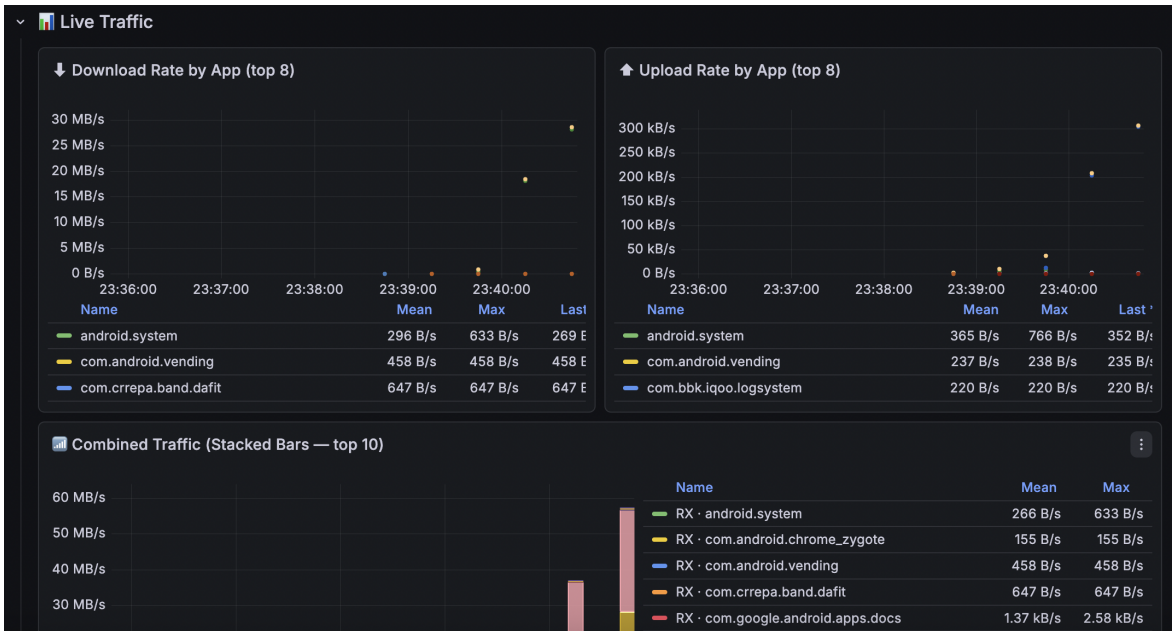


Figure 3: Live traffic time-series — Download Rate by App (top 8) and Upload Rate by App (top 8)

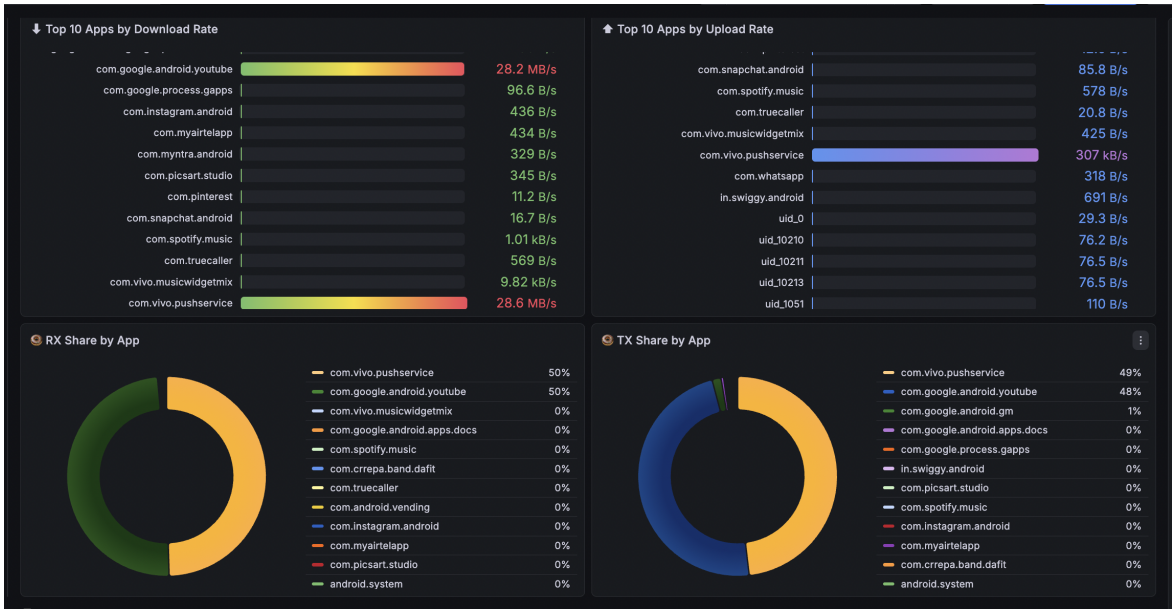


Figure 4: Top 10 apps by download rate (bar gauge) and RX share donut chart

10.4 Popular Apps Row

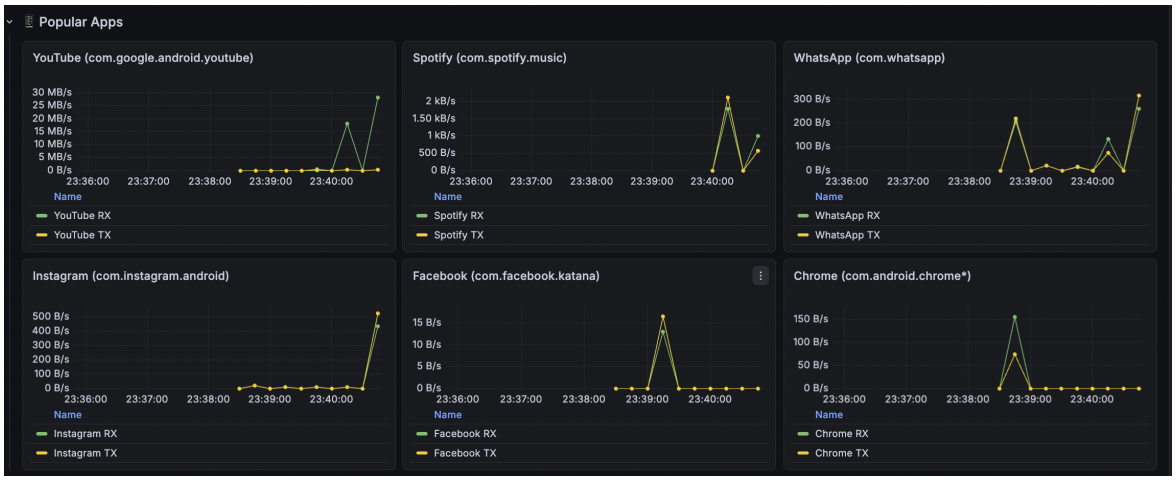
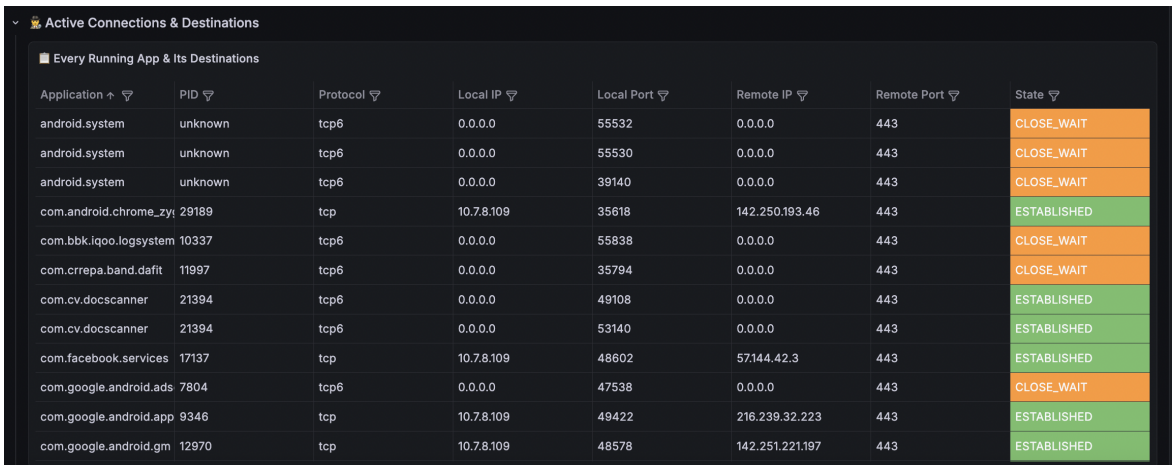


Figure 5: Popular Apps row showing individual RX/TX panels for YouTube, Spotify, WhatsApp, Instagram, Facebook, and Chrome

10.5 Active Connections Table



Application	PID	Protocol	Local IP	Local Port	Remote IP	Remote Port	State
android.system	unknown	tcp6	0.0.0.0	55532	0.0.0.0	443	CLOSE_WAIT
android.system	unknown	tcp6	0.0.0.0	55530	0.0.0.0	443	CLOSE_WAIT
android.system	unknown	tcp6	0.0.0.0	39140	0.0.0.0	443	CLOSE_WAIT
com.android.chrome_zy	29189	tcp	10.7.8.109	35618	142.250.193.46	443	ESTABLISHED
com.bbk.iqoo.logsystem	10337	tcp6	0.0.0.0	55838	0.0.0.0	443	CLOSE_WAIT
com.crrepa.band.dafit	11997	tcp6	0.0.0.0	35794	0.0.0.0	443	CLOSE_WAIT
com.cv.docscanner	21394	tcp6	0.0.0.0	49108	0.0.0.0	443	ESTABLISHED
com.cv.docscanner	21394	tcp6	0.0.0.0	53140	0.0.0.0	443	ESTABLISHED
com.facebook.services	17137	tcp	10.7.8.109	48602	57.144.42.3	443	ESTABLISHED
com.google.android.ads	7804	tcp6	0.0.0.0	47538	0.0.0.0	443	CLOSE_WAIT
com.google.android.app	9346	tcp	10.7.8.109	49422	216.239.32.223	443	ESTABLISHED
com.google.android.gm	12970	tcp	10.7.8.109	48578	142.251.221.197	443	ESTABLISHED

Figure 6: Active Connections table showing Application, PID, Protocol, Local IP, Local Port, Remote IP, Remote Port, and State columns with colour-coded state indicators

11. Key PromQL Queries Used in the Dashboard

The following PromQL expressions power the dashboard panels:

```
# Total device download rate (bytes/s)
sum(irate(app_rx_bytes[2m]))

# Top 8 apps by current download rate
```

```
topk(8, sum by (app) (irate(app_rx_bytes[2m])) > 0)

# Per-app RX for a specific application (e.g. YouTube)
irate(app_rx_bytes{app="com.google.android.youtube"}[2m])

# Chrome including all sandboxed sub-processes
sum(irate(app_rx_bytes{app=~"com.android.chrome.*"}[2m]))

# Count of ESTABLISHED connections per app
sum by (app) (app_connections{state="ESTABLISHED"})

# All connections to HTTPS servers
app_connections{rem_port="443"}

# Scrape freshness (should stay below 90 s)
time() - network_exporter_last_scrape_timestamp_seconds
```

Listing 12: PromQL queries used in the Grafana dashboard

The `irate()` function computes an instantaneous rate of change from the last two data points within the specified time window. It is preferred over `rate()` for real-time dashboards because it reacts faster to sudden traffic spikes.

12. Observations

The following observations were made from the experimental results:

1. **YouTube** generated the highest sustained RX traffic due to adaptive bitrate video streaming. Traffic showed a characteristic buffer-fill pattern.
2. **Spotify** generated low to moderate RX traffic. Audio streaming consumes approximately 300–500 KB per track segment at high quality settings, far less than video.
3. **WhatsApp** generated symmetric, low-volume RX and TX spikes. Persistent XMPP connections on port 5222 were visible in the connections table even during idle periods.
4. **Instagram** generated large bursty RX traffic correlated with feed interactions. TX traffic was minimal in receive-only mode.

5. **Facebook** showed a persistent TX baseline even while idle, consistent with background telemetry activity.

6. **Chrome** showed sharp, event-driven spikes tied to page navigation events, with clean return to zero between events.

7. The connections table correctly identified **port 443** (HTTPS) as the dominant destination port across all applications, with WhatsApp additionally using **port 5222** (XMPP).

8. The system confirmed **865 concurrent processes** on the test device, demonstrating the scale of background activity on a modern Android phone.

9. The UID-based connection mapping worked correctly in all test cases, with no misattributions observed between applications sharing similar traffic patterns.

13. Advantages

- **No root required:** The system uses `dumpsys netstats` and the UID column in `/proc/net/tcp`, both accessible without elevated privileges.

- **Accurate UID-to-app mapping:** The three-layer UID dictionary (system UIDs, `pm list packages`, `ps -A`) ensures correct attribution for both system services and user-installed applications.

- **Real-time visibility:** The 30-second scrape interval and 15-second dashboard refresh provide near-real-time traffic visibility.

- **Application-level granularity:** Traffic is attributed to specific package names (e.g., `com.google.android.youtube`) rather than just interface totals.

- **Connection-level detail:** The connections table exposes the full 5-tuple (local IP, local port, remote IP, remote port, state) for every active TCP socket.

- **Open-source stack:** All components — Python, Prometheus, Grafana, Docker — are freely available and well-documented.

- **Auto-provisioned:** The Grafana dashboard and Prometheus datasource are provisioned automatically from configuration files, requiring zero manual setup.

- **Extensible:** New metrics (e.g., UDP connections, DNS queries, packet counts) can be added to the exporter without modifying Prometheus or Grafana configuration.

14. Limitations

- **Cumulative counters only:** `dumpsys netstats` provides bytes-since-boot counters. The `irate()` function in Prometheus derives rates from these counters, so the first data point after an application launches may undercount traffic.
- **ADB dependency:** The system requires a physical or wireless ADB connection. If the USB cable is disconnected, the exporter stops receiving data.
- **Some Android versions restrict `/proc` access:** Newer Android versions (12+) may limit `/proc/net/tcp` visibility to root processes. In such cases, connection attribution may fall back to UID-only labels without full socket detail.
- **No UDP tracking:** The current implementation only reads TCP connections from `/proc/net/tcp`. UDP traffic (used by DNS, QUIC/HTTP3, and some streaming protocols) is not attributed per-connection.
- **ADB throughput overhead:** Each 30-second collection cycle issues four sequential ADB commands. On slower USB connections or busy devices, this can take 10–20 seconds, leaving only 10–20 seconds of idle time between cycles.
- **Isolated process attribution:** Android sandboxes certain operations in isolated processes (UID prefix `u0_i`). These are labelled as `android.isolated_sandbox` rather than their parent application.

15. Future Scope

- **eBPF-based monitoring:** Replacing ADB-based collection with eBPF programs attached to the network stack would give packet-level visibility with sub-millisecond latency.
- **UDP and QUIC support:** Adding `/proc/net/udp` parsing would capture DNS queries, QUIC connections (used by YouTube and Google services), and other UDP-based traffic.
- **Alerting:** Prometheus Alertmanager could send notifications when an application exceeds a configurable bandwidth threshold, enabling automated data usage warnings.
- **AI-based anomaly detection:** Integrating a machine learning model to establish baseline traffic profiles for each application and flag statistical anomalies could help detect malware or data exfiltration.

- **Wireless ADB:** Removing the USB cable dependency by using ADB over TCP would allow monitoring of devices in normal usage conditions.
- **Multi-device support:** Extending the exporter to manage multiple ADB-connected devices simultaneously would enable fleet-level monitoring.

16. Conclusion

This project successfully designed and implemented a real-time, application-level Network Data Analyzer for Android devices. The system achieves its core objectives: per-application traffic monitoring without root access, accurate UID-to-app-name resolution, TCP connection attribution, real-time Prometheus metrics, and comprehensive Grafana visualisation.

The key technical contributions are:

1. A **no-root data collection method** using `dumpsys netstats --uid` as the primary traffic source.
2. A **three-layer UID resolution algorithm** that combines system UIDs, `pm list packages`, and `ps -A` for accurate application attribution.
3. A **zero-overhead connection mapping technique** that reads the UID column directly from `/proc/net/tcp`, eliminating the need for root-level file descriptor scanning.
4. A **fully auto-provisioned monitoring stack** using Docker Compose, Prometheus, and Grafana that requires a single command to deploy.

Experimental results validated that the system correctly identifies and quantifies the distinct traffic patterns of different application categories: sustained streaming (YouTube), periodic buffering (Spotify), event-driven browsing (Chrome), and low-volume persistent messaging (WhatsApp). The Grafana dashboard provides clear, real-time evidence of these patterns through time-series graphs, ranking gauges, donut charts, and a fully filterable connections table.

The system demonstrates that deep network observability on Android is achievable with open-source tools and standard debugging interfaces, without requiring device modification.

17. References

1. Android Debug Bridge (ADB) — Android Developers Documentation. https://developer.android.com/tools/adb
2. Android Network Statistics — `dumpsys netstats` Internals. Android Open Source Project.
3. Linux `/proc/net/tcp` — The Linux Kernel Documentation. https://www.kernel.org/doc/html/latest/networking/
4. Prometheus Data Model and Exposition Format. https://prometheus.io/docs/instrumenting/exposition_formats/
5. PromQL — Prometheus Query Language Reference. https://prometheus.io/docs/prometheus/latest/querying/basics/
6. Grafana Dashboard Provisioning. https://grafana.com/docs/grafana/latest/administration/provisioning/
7. Docker Compose Documentation. https://docs.docker.com/compose/
8. Android UID and Permission Model — Android Developers. https://developer.android.com/guide/topics/permissions/overview
9. TCP Connection States (RFC 793) — Transmission Control Protocol. IETF.